SC4064 GPU Programming

Assignment 3

Instructor: Chen Wang
chen.wang@ntu.edu.sg
College of Computing and Data Science
Nanyang Technological University

Objectives

This is the last assignment of the SC4064 GPU Programming course. You will design and implement a GPU-accelerated image processing pipeline that brings together everything covered in the course.

1. Apply shared memory tiling with halo cells to a real 2D convolution problem.
2. Implement parallel histogram computation using atomic operations.
3. Use CUDA streams and asynchronous transfers to pipeline computation across stages.
4. Distribute a workload across two GPUs.
5. Profile a non-trivial GPU application using Nsight Systems.
6. Apply the roofline model to reason about kernel performance.

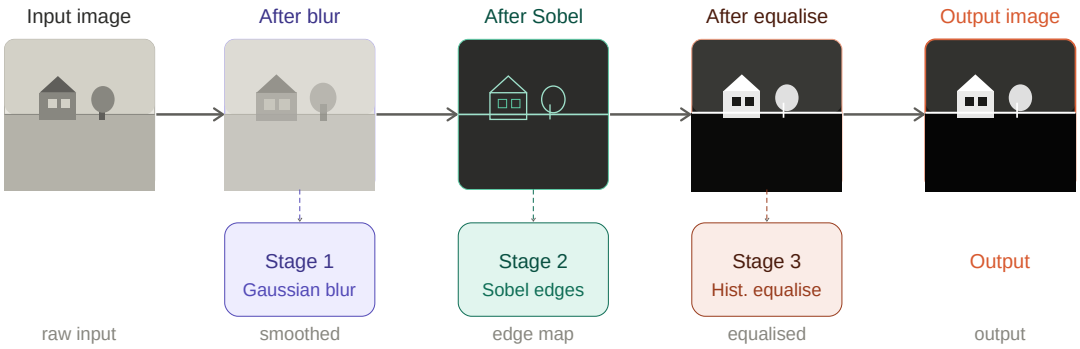
Platform

The CUDA programs in this assignment are designed to run on NVIDIA GPUs. At NSCC Singapore, the ASPIRE 2A supercomputer provides access to NVIDIA A100 GPUs, which we encourage you to use for this assignment. You may also use your personal computer or workstation if it has a working CUDA driver and CUDA Toolkit installed. Please document your computing environment if you are not using ASPIRE 2A.

1 Overview

The image processing pipeline in this assignment mirrors a popular workload in computer vision, medical imaging, etc, where a large batch of raw images need to be transformed through a fixed sequence of operations.

The three stages in this assignment include: a smoothing pass (Stage 1 - Gaussian blur) suppresses noise so that the subsequent analysis is not misled by pixel-level artifacts; an edge detection pass (Stage 2 - Sobel) extracts structural features; and a contrast normalisation pass (Stage 3 - histogram equalisation) redistributes intensity values so that the output is perceptually useful regardless of how the original image was exposed.



2 Background

This section introduces the image processing concepts. You are not expected to have prior knowledge of these topics. Read this section carefully before starting.

2.1 Greyscale Images and Pixel Representation

A greyscale image is a 2D array of pixels, where each pixel stores a single intensity value between 0 (black) and 255 (white). An image of width W and height H is stored in memory as a flat array of $W \times H$ unsigned 8-bit integers (`uint8_t`), in row-major order, meaning pixel at column x, row y is at index $y \times W + x$.

All images in this assignment are 256×256 pixels in PGM (Portable Greymap) format. A PGM file is a plain text header followed by the raw pixel data. You do not need to understand the file format. A minimal PGM file loader and saver are provided.

2.2 Gaussian Blur

Gaussian blur is a spatial smoothing operation that reduces noise and fine detail in an image. Each output pixel is computed as a weighted average of its neighbouring input pixels, where the weights follow a 2D Gaussian (bell-curve) distribution — neighbours closer to the centre contribute more than those further away.

The operation is expressed as a 2D convolution: slide a small weight matrix (called a kernel or filter) across the image, and at each position, multiply each overlapping pixel by the corresponding weight and sum the results. For a 5×5 Gaussian kernel, each output pixel depends on a 5×5 neighbourhood of input pixels. The resulting sum is then clamped to the range [0,255]:

```
output = (uint8_t)min(max((int)roundf(sum), 0), 255);
```

Note: `roundf` is required to match the reference image.

Convolution is memory-bound: each output pixel requires reading 25 input pixels, but neighbouring output pixels share most of those reads. Naively reading from global memory means the same input pixel is fetched many times by different threads. Shared memory tiling exploits this reuse to reduce global memory traffic.

2.3 Halo Cells (Ghost Cells)

When threads in a block collaborate to compute a tile of output pixels, each thread needs access to a neighbourhood of input pixels (not just the pixel directly beneath it). Threads near the edge of a tile need input pixels that belong to an adjacent tile. These extra pixels, i.e., loaded into shared memory but not directly computed by any thread in the block, are called halo cells or ghost cells. For a 5×5 convolution kernel (radius 2), a tile of 16×16 threads needs to load a shared memory region of $(16 + 4) \times (16 + 4) = 20 \times 20$ pixels: the 16×16 centre that threads compute, plus a 2-pixel border on every side.

One key question you need to answer is: which threads are responsible for loading which halo pixels? Since there are more halo pixels than threads in some cases, some threads must load more than one pixel. After all halo pixels are loaded, threads must synchronise with `__syncthreads()` before any computation begins, otherwise a thread might read a halo pixel that a neighbour has not yet written.

2.4 Sobel Edge Detection

Edge detection identifies boundaries between regions of different intensity. The Sobel operator estimates the gradient of pixel intensity at each location: where the gradient magnitude is large, there is a sharp transition in brightness → likely an edge.

The Sobel operator uses two 3×3 kernels (convolutional kernels, not CUDA kernels): one (G_x) that is sensitive to horizontal changes in intensity, and one (G_y) that is sensitive to vertical changes. At each pixel, both kernels are applied by convolution, and the gradient magnitude is computed as $sqrt(G_x^2 + G_y^2)$, clamped to $[0, 255]$. High values in the output indicate strong edges; low values indicate flat regions. The Sobel kernel is computationally similar to Gaussian blur : also a 2D convolution but smaller (3×3 vs 5×5).

2.5 Histogram Equalisation

Histogram equalisation is a contrast-enhancement technique. It redistributes pixel intensities so that the full [0, 255] range is used as evenly as possible, making dark images brighter and revealing detail in underexposed regions.

The process has three steps. First, compute a histogram: count how many pixels have each intensity value (0–255). Second, compute the cumulative distribution function (CDF) of the histogram — a running total of pixel counts from intensity 0 up to each level. Third, remap each pixel using the formula:

$$new\_val = round((CDF[old\_val] - CDF_{min})/(W \times H - CDF_{min}) \times 255)$$

where CDF_min is the smallest non-zero CDF value. This formula maps the original intensity distribution onto the full range, effectively stretching the histogram.

On the GPU, step one (the histogram) can be computed using atomic operations, since many threads may try to increment the same intensity counter simultaneously. Step two (the CDF) is a prefix sum (scan), which you may compute on the CPU side using `thrust::exclusive_scan`. Step three is a straightforward remapping kernel.

2.6 The Roofline Model

We discussed the roofline model in class; here's a quick recap. The roofline model is a visual and analytical tool for understanding whether a kernel is limited by compute throughput or memory bandwidth.

The key concept is *arithmetic intensity*: the ratio of floating-point operations performed to bytes transferred from memory, measured in FLOPs/byte. A kernel with low arithmetic intensity (e.g., a simple copy or a blur) performs few operations per byte fetched, so it is memory-bound. A kernel with high arithmetic intensity (e.g., matrix multiplication) can keep the compute units busy even if memory is slow, so it may be compute-bound.

On a roofline plot, the x-axis is arithmetic intensity and the y-axis is achieved performance (GFLOPs/s). Two lines form the "roof": a horizontal line at peak compute throughput, and a diagonal line rising from the origin at a slope equal to peak memory bandwidth. The roofline for a kernel sits at the lower of these two limits. The ridge point (the intersection of the two lines) marks the arithmetic intensity at which the kernel transitions from memory-bound to compute-bound.

In your report you will calculate the arithmetic intensity of each kernel, identify where each sits on the roofline, and compare your achieved performance against the theoretical limits. This tells you whether your optimisation effort should focus on reducing memory traffic (if memory-bound) or increasing compute efficiency (if compute-bound).

3 Implementation (5 Marks)

You will implement all three pipeline stages, a CUDA streams dispatcher, and a multi-GPU batch distributor. The skeleton code, Makefile, and input images are provided (Section 4); you only need to complete the TODOs in the specified source files. Detailed requirements for each component are described below.

3.1 Stage 1 — Gaussian Blur (1 Mark)

Implement `gaussianBlurKernel()` in pipeline.cu using 2D shared memory tiling with halo cells. Apply the 5×5 Gaussian kernel (given in pipeline.cu).

Requirements:

- Declare shared memory with correct halo dimensions: $(TILE\_W + 4) \times (TILE\_H + 4)$ for a 5×5 kernel (radius 2).
- Halo pixels should be loaded cooperatively by all threads in the block.
- Tile dimensions should be configurable via compile-time constants $TILE\_W$ and $TILE\_H$ (default: 16×16).
- Boundary condition: clamp-to-edge. Out-of-bounds pixel accesses should use the value of the nearest valid pixel.

- Output must match the provided reference images within ±2 intensity level (checked automatically).

3.2 Stage 2 — Sobel Edge Detection (1 Mark)

Implement `sobelKernel()` in pipeline.cu. Apply the standard Sobel operator using the two 3×3 kernels below, compute gradient magnitude as $sqrt(G_x^2 + G_y^2)$, and clamp the result to [0, 255].

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad G_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Requirements:

- Both G_x and G_y must be applied in a single kernel launch (not two separate launches).
- Shared memory tiling is optional.
- Boundary condition: clamp-to-edge.
- Output must match the reference within ±2 intensity level (checked automatically).

3.3 Stage 3 — Histogram Equalisation (1 Mark)

Implement histogram equalisation across three sub-steps, each as a separate CUDA kernel or host call.

3.3.1 Step A — Histogram Kernel

Implement `histogramKernel()` that counts the frequency of each intensity value (0–255) across the image. Each thread processes one or more pixels and increments the corresponding histogram bin using `__atomicAdd`.

3.3.2 Step B — CDF

Compute the cumulative distribution function (CDF) of the histogram on the CPU (you may also implement it on the GPU). The solution code for this step is provided; you can either directly use it or implement your own. The result should be a 256-element array where entry i contains the total number of pixels with intensity strictly less than i.

3.3.3 Step C — Equalisation Kernel

Implement `equalizeKernel()` that remaps each pixel using:

$$new\_val = round((CDF[old\_val] - CDF_{min})/(W \times H - CDF_{min}) \times 255)$$

where CDF_{min} is the smallest non-zero value in the CDF array (find it on the host after the scan, or in a brief device kernel).

3.4 CUDA Streams and Multi-GPU Batch Distribution (2 Marks)

3.4.1 CUDA Streams Pipeline (1 Mark)

First, connect all three stages in multigpu.cu using CUDA streams and asynchronous memory transfers so that, where possible, memory transfers for one image overlap with computation on another.

Hint: The simplest correct design: create one stream per image. Launch `cudaMemcpyAsync (H→D)`, `gaussianBlurKernel`, `sobelKernel`, `histogramKernel`, `thrust::exclusive_scan`, `equalizeKernel`, and `cudaMemcpyAsync (D→H)` all on the same stream. Dependencies within a stream are implicit, i.e., kernels execute in launch order. After submitting all images, call `cudaStreamSynchronize` on each stream.

3.4.2 Multi-GPU Batch Distribution (1 Mark)

Finally, in multigpu.cu, extend your pipeline to distribute the image batch across two GPUs. Call `cudaGetDeviceCount` to detect the number of available GPUs. Split the batch evenly across GPU 0 and GPU 1. Call `cudaSetDevice(device_id)` before any CUDA API call made in the context of that GPU. Allocate separate device buffers for each GPU. Gather all results back to the host and write output files in the correct order.

4 Starter Code

Download `assignment3_starter.tar.gz` from NTULearn. Read `README.md` thoroughly. It explains how to modify, build, run, check, and profile your code.

```
assignment3/
  README.md
  Makefile                # Provided. TODO: Fill in CUDA compilation flags
  src/
    main.cu               # Entry point
    pipeline.cu           # TODO: Stages 1, 2, and 3 kernel implementations
    pipeline.cuh          # Kernel declarations; TILE_W, TILE_H constants, etc.
    multigpu.cu           # TODO: CUDA Streams and Multi-GPU Split
    multigpu.cuh          # Multi-GPU interface declarations
    pgm_io.cu             # Provided: PGM loader/saver - do not modify
    pgm_io.cuh
  data/
    input/                # 20 test images (512x512 greyscale PGM)
    expected_output/      # Reference outputs for self-checking
```

5 Performance Analysis & Report (5 marks)

Your report must be no longer than 4 pages. The report is not a log of what you did, it is an analysis of what your GPU did. *Note: when measuring the kernel time, do not include the I/O time (the PGM read/write time).*

5.1 Roofline Analysis (3 marks)

Perform a roofline analysis of all three pipeline kernels (Gaussian blur, Sobel, histogram + equalise). Your analysis must include the following:

- **Arithmetic Intensity**. Calculate the arithmetic intensity (AI) of each kernel in FLOPs/byte. Count the number of floating-point operations per output pixel. Be precise: additions, multiplications, and divisions each count as one FLOP. Conversions and address calculations do not count. Count the number of bytes transferred from global memory per output pixel.
- **Roofline Plot**. Draw a roofline plot (by hand or with a tool) for your GPU. Mark the ridge point. Plot each of your three kernels as a point at (AI, achieved GFLOPs/s).
- **Interpretation**. For each kernel, answer: Is the kernel memory-bound or For the Gaussian blur: how much did shared memory tiling reduce global memory traffic compared to a naive implementation? Which kernel is the pipeline bottleneck? What would need to change to improve overall throughput?

5.2 Stream Overlap Analysis (1 mark)

Profile your stream pipeline using Nsight Systems. Answer the following: Is there visible overlap between H→D memory transfers and kernel execution across images? If you observe no overlap, explain why. Is it a pinned memory issue, a stream design issue, or a hardware constraint? What specific change would you make to achieve overlap?

5.3 Multi-GPU Speedup Analysis (1 mark)

Measure total end-to-end time (excluding I/O) to process all 20 images using (a) one GPU and (b) two GPUs. What speedup did you achieve? The theoretical maximum is 2× how close did you get, and what limited you? If you had access to 4 GPUs, what speedup would Amdahl's Law predict? What practical factors (beyond the law's assumptions) would further limit performance?

6 Submission

Please compress all files (code, report, etc) into a single archive (.zip or .tar.gz) and submit it via NTULearn by the specified deadline. Late submissions will **not** be accepted.

Checklist:

- `make build` compiles cleanly from a fresh directory (no errors).
- `make run` produces 20 output PGM files
- `make check` pass the automated diff check.
- Report addresses every point in Section 5.

Marking Scheme

Implementation	**Marks**	**Evidence**
Stage 1: Gaussian blur kernel	1	Code + automated output check
Stage 2: Sobel edge detection kernel	1	Code + automated output check
Stage 3: Histogram equalisation	1	Code + automated output check
CUDA streams pipeline	1	Code
Multi-GPU distribution	1	Code
Performance Analysis	**Marks**	**Requirement**
Roofline analysis	3	Report
Stream overlap analysis	1	Report
Multi-GPU Speedup analysis	1	Report
Total	**10**	